

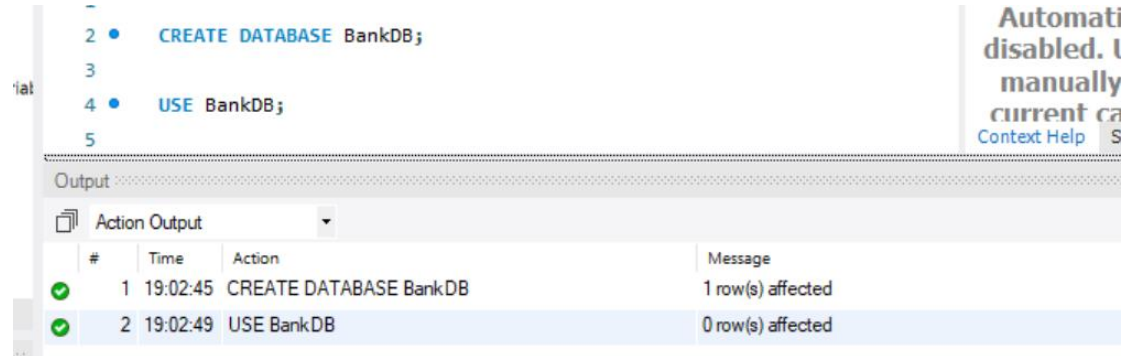
Database System Engineering and Distributed Backend Development
Course Code: 25CS1302E

2520030198

K.Vaishnavi

TRIGGERS AND STORED PROCEDURES

1. CREATE DATABASE



The screenshot shows a SQL query window with the following commands:

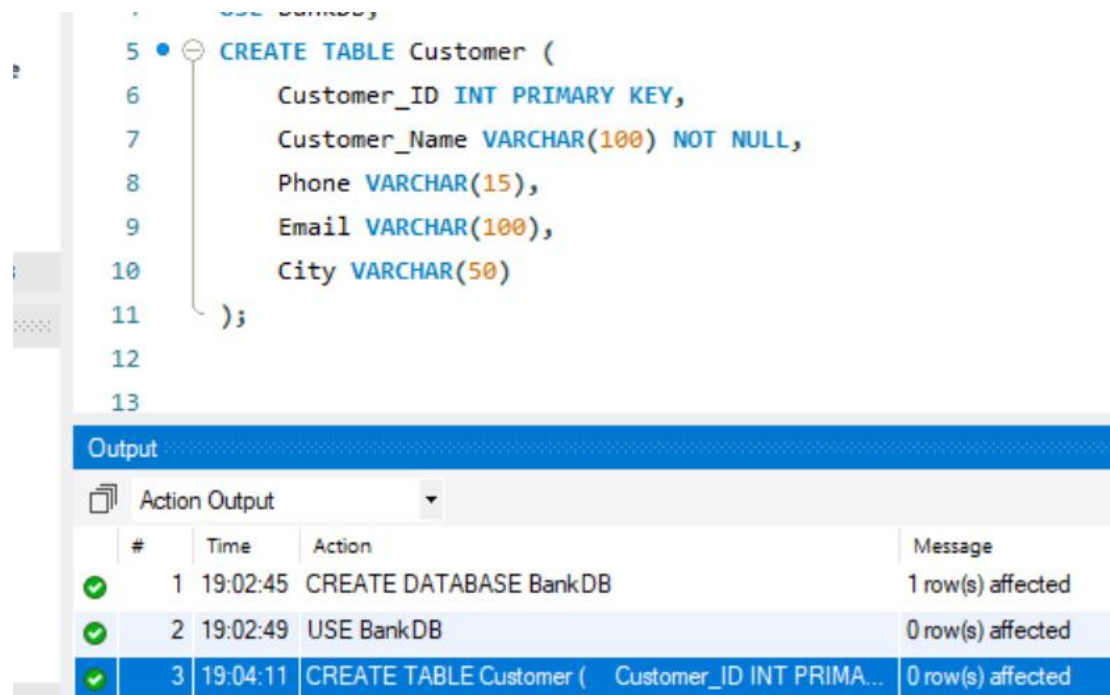
```
2 • CREATE DATABASE BankDB;  
3  
4 • USE BankDB;  
5
```

On the right, a message box states: "Automatic save is disabled. You must manually save the current query." with links for "Context Help" and "Save".

The Output window shows the "Action Output" for the executed commands:

#	Time	Action	Message
✓ 1	19:02:45	CREATE DATABASE BankDB	1 row(s) affected
✓ 2	19:02:49	USE BankDB	0 row(s) affected

2. CREATE CUSTOMER TABLE



The screenshot shows a SQL query window with the following commands:

```
5 • CREATE TABLE Customer (  
6     Customer_ID INT PRIMARY KEY,  
7     Customer_Name VARCHAR(100) NOT NULL,  
8     Phone VARCHAR(15),  
9     Email VARCHAR(100),  
10    City VARCHAR(50)  
11 );  
12  
13
```

The Output window shows the "Action Output" for the executed commands:

#	Time	Action	Message
✓ 1	19:02:45	CREATE DATABASE BankDB	1 row(s) affected
✓ 2	19:02:49	USE BankDB	0 row(s) affected
✓ 3	19:04:11	CREATE TABLE Customer (Customer_ID INT PRIMA...	0 row(s) affected

3. CREATE ACCOUNT TABLE

```

12 • CREATE TABLE Account (
13     Account_No INT PRIMARY KEY,
14     Customer_ID INT,
15     Account_Type VARCHAR(20),
16     Balance DECIMAL(12,2) DEFAULT 0,
17     Branch VARCHAR(50),
18
19     FOREIGN KEY (Customer_ID)
20     REFERENCES Customer(Customer_ID)
21 );
22
23
24

```

Output

Action Output				
#	Time	Action	Message	
✓ 2	19:02:49	USE BankDB	0 row(s) affected	
✓ 3	19:04:11	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected	
✓ 4	19:13:06	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected	

4. CREATE TRANSACTION TABLE

```

22 • CREATE TABLE Bank_Transaction (
23     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
24     Account_No INT,
25     Transaction_Type VARCHAR(20),
26     Amount DECIMAL(12,2),
27     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
28
29     FOREIGN KEY (Account_No)
30     REFERENCES Account(Account_No)
31 );
32

```

Output

Action Output				
#	Time	Action	Message	
✓ 3	19:04:11	CREATE TABLE Customer (Customer_ID INT PRIM...	0 row(s) affected	
✓ 4	19:13:06	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected	
✓ 5	19:13:56	CREATE TABLE Bank_Transaction (Transaction_I...	0 row(s) affected	

5. CREATE LOAN TABLE

```

32 • CREATE TABLE Loan (
33     Loan_ID INT PRIMARY KEY,
34     Customer_ID INT,
35     Loan_Type VARCHAR(30),
36     Loan_Amount DECIMAL(12,2),
37     Interest_Rate DECIMAL(5,2),
38
39     FOREIGN KEY (Customer_ID)
40     REFERENCES Customer(Customer_ID)
41 );
42

```

Output

Action Output				
	#	Time	Action	Message
✓	4	19:13:06	CREATE TABLE Account (Account_No INT PRIMA...	0 row(s) affected
✓	5	19:13:56	CREATE TABLE Bank_Transaction (Transaction_I...	0 row(s) affected
✓	6	19:14:30	CREATE TABLE Loan (Loan_ID INT PRIMARY KE...	0 row(s) affected

6. INSERT CUSTOMER DATA

```

44 • INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City)
45 VALUES
46     (101, 'Vaishnavi', '9876543210', 'vaishnavi@gmail.com', 'Hyderabad')
47     (102, 'likhitha', '9876543211', 'likhitha@gmail.com', 'Vijayawada'),
48     (103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),
49     (104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),
50     (105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');
51
52

```

toggle automatic

Context Help Snippets

Output

Action Output

	#	Time	Action	Message	Duration / Fet
✓	16	19:26:29	CREATE TABLE Loan (Loan_ID INT PRIMARY KE...	0 row(s) affected	0.063 sec
✓	17	19:26:37	INSERT INTO Customer (Customer_ID, Customer_Nam...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.015 sec

7. INSERT ACCOUNT DATA

52 • **INSERT INTO Account**
 53 (Account_No, Customer_ID, Account_Type, Balance, Branch)
 54 **VALUES**
 55 (10001, 101, 'Savings', 50000, 'Hyderabad'),
 56
 57 (10002, 102, 'Savings', 75000, 'Vijayawada'),
 58
 59 (10003, 103, 'Current', 120000, 'Bangalore'),
 60
 61 (10004, 104, 'Savings', 45000, 'Chennai'),
 62
 63 (10005, 105, 'Current', 90000, 'Hyderabad');
 64
 65

Output

Action Output

#	Time	Action	Message
✓ 17	19:26:37	INSERT INTO Customer (Customer_ID, Customer_Nam...	5 row(s) affected Records: 5 Duplicates: 0
✓ 18	19:27:09	INSERT INTO Account (Account_No, Customer_ID, A...	5 row(s) affected Records: 5 Duplicates: 0

8. INSERT TRANSACTION DATA

65 • **INSERT INTO Bank_Transaction**
 66 (Account_No, Transaction_Type, Amount)
 67 **VALUES**
 68 (10001, 'DEPOSIT', 10000),
 69
 70 (10002, 'DEPOSIT', 15000),
 71
 72 (10003, 'WITHDRAW', 20000),
 73
 74 (10004, 'DEPOSIT', 5000),
 75
 76 (10005, 'WITHDRAW', 10000);
 77

Output

Action Output

#	Time	Action	Message
✓ 18	19:27:09	INSERT INTO Account (Account_No, Customer_ID, A...	5 row(s) affected Records: 5 Duplicates: 0
✓ 19	19:27:38	INSERT INTO Bank_Transaction (Account_No, Trans...	5 row(s) affected Records: 5 Duplicates: 0

9. INSERT LOAN DATA


```

78 • INSERT INTO Loan
79 (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
80 VALUES
81 (501, 101, 'Home Loan', 500000, 7.5),
82
83 (502, 102, 'Education Loan', 100000, 6.5),
84
85 (503, 103, 'Car Loan', 800000, 8.2),
86
87 (504, 104, 'Personal Loan', 500000, 10.5);
88

```

Output				
Action Output				
#	Time	Action	Message	
19	19:27:38	INSERT INTO Bank_Transaction (Account_No, Trans...	5 row(s) affected Records: 5 Duplicates: 0	
20	19:28:06	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Ty...	4 row(s) affected Records: 4 Duplicates: 0	

10. DISPLAY TABLE DATA

```

89
90 • SELECT * FROM Customer;
91
92 • SELECT * FROM Account;

```

Customer_ID	Customer_Name	Phone	Email	City
101	Vaishnavi	9876543210	vaishnavi@gmail.com	Hyderabad
102	likhitha	9876543211	likhitha@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

```

92 • SELECT * FROM Account;
93
94

```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

94

95 • `SELECT * FROM Bank_Transaction;`

96

97 • `SELECT * FROM Loan;`

Result Grid | Filter Rows: | Edit: | Export/Import:

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	1	10001	DEPOSIT	10000.00	2026-08-16 19:27:38
	2	10002	DEPOSIT	15000.00	2026-08-16 19:27:38
	3	10003	WITHDRAW	20000.00	2026-08-16 19:27:38
	4	10004	DEPOSIT	5000.00	2026-08-16 19:27:38
	5	10005	WITHDRAW	10000.00	2026-08-16 19:27:38
•	NULL	NULL	NULL	NULL	NULL

97 • `SELECT * FROM Loan;`

98

99

Result Grid | Filter Rows: | Edit: | Export/Import:

	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	501	101	Home Loan	5000000.00	7.50
	502	102	Education Loan	1000000.00	6.50
	503	103	Car Loan	800000.00	8.20
	504	104	Personal Loan	500000.00	10.50
•	NULL	NULL	NULL	NULL	NULL

11. PROCEDURE TO DISPLAY ALL CUSTOMERS

99

100 • `DELIMITER //`

101 • `CREATE PROCEDURE GetAllCustomers()`

102 • `BEGIN`

103 • `SELECT * FROM Customer;`

104 • `END //`

105 • `CALL GetAllCustomers();`

106

107

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | **Result Grid**

	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Vaishnavi	9876543210	vaishnavi@gmail.com	Hyderabad
	102	likhitha	9876543211	likhitha@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

Result 7 x | Read Only | Context Help

Output

Action Output

#	Time	Action	Message
35	20:03:28	CREATE PROCEDURE GetAllCustomers() BEGIN	0 row(s) affected
36	20:03:34	CALL GetAllCustomers()	5 row(s) returned

Session

Automata disabled. manually current toggle

12. PROCEDURE TO DISPLAY ACCOUNT DETAILS FOR A GIVEN ACCOUNT NUMBER

```

109 CREATE PROCEDURE GetAccountDetails(
110     IN p_Account_No INT
111 )
112 BEGIN
113     SELECT *
114     FROM Account
115     WHERE Account_No = p_Account_No;
116 END //
117 DELIMITER ;
118 CALL GetAccountDetails(10001);

```

Result Grid

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

12. PROCEDURE TO FIND CUSTOMER ACCOUNTS

```

121 CREATE PROCEDURE GetCustomerAccounts(
122     IN p_Customer_ID INT
123 )
124 BEGIN
125
126     SELECT
127         C.Customer_ID,
128         C.Customer_Name,
129         A.Account_No,
130         A.Account_Type,
131         A.Balance,
132         A.Branch
133     FROM Customer C
134     JOIN Account A
135     ON C.Customer_ID = A.Customer_ID
136     WHERE C.Customer_ID = p_Customer_ID;
137
138 END //
139
140 DELIMITER ;

```

Output

#	Time	Action	Message
42	20:08:03	CALL GetAccountDetails(10001)	1 row(s) returned
43	20:10:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN ...	0 row(s) affected

```

141 CALL GetCustomerAccounts(101);
142

```

Result Grid

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Vaishnavi	10001	Savings	50000.00	Hyderabad

13. PROCEDURE TO DEPOSIT MONEY

```

145 CREATE PROCEDURE DepositMoney(
146     IN p_Account_No INT,
147     IN p_Amount DECIMAL(12,2)
148 )
149 BEGIN
150
151     UPDATE Account
152     SET Balance = Balance + p_Amount
153     WHERE Account_No = p_Account_No;
154
155 END //
156
157 DELIMITER ;
158 CALL DepositMoney(10001, 5000);
159
160

```

Output

#	Time	Action	Message
43	20:10:01	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN ...	0 row(s) affected
44	20:10:37	CALL GetCustomerAccounts(101)	1 row(s) returned
45	20:11:07	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DE...	0 row(s) affected
46	20:11:15	CALL DepositMoney(10001, 5000)	1 row(s) affected

14. PROCEDURE TO WITHDRAW MONEY

```

164 CREATE PROCEDURE WithdrawMoney(
165     IN p_Account_No INT,
166     IN p_Amount DECIMAL(12,2)
167 )
168 BEGIN
169
170     UPDATE Account
171     SET Balance = Balance - p_Amount
172     WHERE Account_No = p_Account_No;
173
174 END //
175
176 DELIMITER ;
177
178 CALL WithdrawMoney(10001, 3000);
179
180

```

Output

#	Time	Action	Message
46	20:11:15	CALL DepositMoney(10001, 5000)	1 row(s) affected
47	20:12:13	CALL WithdrawMoney(10001, 3000)	Error
48	20:12:17	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount D...	0 row(s) affected
49	20:12:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected

TRIGGERS

16. TRIGGER – PREVENT NEGATIVE BALANCE

```

182 CREATE TRIGGER CheckBalance
183 BEFORE UPDATE ON Account
184 FOR EACH ROW
185 BEGIN
186
187     IF NEW.Balance < 0 THEN
188
189         SIGNAL SQLSTATE '45000'
190         SET MESSAGE_TEXT =
191             'Transaction failed: Insufficient balance';
192
193     END IF;
194
195 END //
196
197 DELIMITER ;
198
199 UPDATE Account
200 SET Balance = Balance - 60000
201 WHERE Account_No = 10001;

```

Output

#	Time	Action	Message
✓ 49	20:12:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
✓ 50	20:15:06	UPDATE Account SET Balance = Balance - 60000 WHERE Account_No = 10001	1 row(s) affected Rows
✓ 51	20:15:12	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW ...	0 row(s) affected
✗ 52	20:15:15	UPDATE Account SET Balance = Balance - 60000 WHERE Account_No = 10001	Error Code: 1644. Tran

17. TRIGGER – PREVENT NEGATIVE DEPOSIT

```

205
206 CREATE TRIGGER CheckTransactionAmount
207 BEFORE INSERT ON Bank_Transaction
208 FOR EACH ROW
209 BEGIN
210
211     IF NEW.Amount <= 0 THEN
212
213         SIGNAL SQLSTATE '45000'
214         SET MESSAGE_TEXT =
215             'Transaction amount must be greater than zero';
216
217     END IF;
218
219 END //
220
221 DELIMITER ;
222
223 INSERT INTO Bank_Transaction
224 (Account_No, Transaction_Type, Amount)
225 VALUES

```

Output

#	Time	Action	Message
✗ 52	20:15:15	UPDATE Account SET Balance = Balance - 60000 WHERE Account_No = 10001	Error Code: 1644. Transaction f
✓ 53	20:16:32	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (...	1 row(s) affected
✓ 54	20:16:42	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transactio...	0 row(s) affected
✗ 55	20:16:44	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (...	Error Code: 1644. Transaction a

18. CREATE TRANSACTION AUDIT TABLE

```

226
227 • CREATE TABLE Transaction_Audit (
228     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
229     Transaction_ID INT,
230     Account_No INT,
231     Transaction_Type VARCHAR(20),
232     Amount DECIMAL(12,2),
233     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
234 );
235
236
237
238
239

```

Output

Action Output

#	Time	Action	Message
53	20:16:32	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (...)	1 row(s) affected
54	20:16:42	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction...	0 row(s) affected
55	20:16:44	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (...)	Error Code: 1644. Transaction am...
56	20:18:14	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREM...	0 row(s) affected

19. TRIGGER – TRANSACTION AUDIT

```

DELIMITER //
CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
    (
        Transaction_ID,
        Account_No,
        Transaction_Type,
        Amount
    )
    VALUES
    (
        NEW.Transaction_ID,
        NEW.Account_No,
        NEW.Transaction_Type,
        NEW.Amount
    );
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 2500);
SELECT * FROM Transaction_Audit;

```

20. TRIGGER – AUTOMATICALLY UPDATE ACCOUNT BALANCE

```

DELIMITER //
CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'DEPOSIT' THEN
        UPDATE Account
        SET Balance = Balance + NEW.Amount
        WHERE Account_No = NEW.Account_No;
    ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
        UPDATE Account
        SET Balance = Balance - NEW.Amount
        WHERE Account_No = NEW.Account_No;
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 5000);
SELECT *
FROM Account
WHERE Account_No = 10001;

```

Result Grid					
		Filter Rows:		Edit:	
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	-8000.00	Hyderabad
*	NULL	NULL	NULL	NULL	NULL

21. TRIGGER – PREVENT WITHDRAWAL ABOVE BALANCE

```

DELIMITER //
CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    DECLARE CurrentBalance DECIMAL(12,2);
    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;
    IF NEW.Transaction_Type = 'WITHDRAW'
        AND NEW.Amount > CurrentBalance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Withdrawal failed: Insufficient balance';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);

```

✖	71	20:27:03	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (...)	Error Code: 1644. Withdrawal fail
---	----	----------	--	-----------------------------------

22. PROCEDURE – ACCOUNT TRANSFER


```

DELIMITER //
CREATE PROCEDURE TransferMoney(
    IN SenderAccount INT,
    IN ReceiverAccount INT,
    IN TransferAmount DECIMAL(12,2)
)
BEGIN
    DECLARE SenderBalance DECIMAL(12,2);
    SELECT Balance
    INTO SenderBalance
    FROM Account
    WHERE Account_No = SenderAccount;
    IF SenderBalance < TransferAmount THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Transfer failed: Insufficient balance';
    ELSE
        UPDATE Account
        SET Balance = Balance - TransferAmount
        WHERE Account_No = SenderAccount;
        UPDATE Account
        SET Balance = Balance + TransferAmount
        WHERE Account_No = ReceiverAccount;
    END IF;
END //
DELIMITER ;
CALL TransferMoney(10001, 10002, 5000);
SELECT *
FROM Account
WHERE Account_No IN (10001,10002);

```

Result Grid					
		Filter Rows:		Edit:	
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	-8000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
*	NULL	NULL	NULL	NULL	NULL

QUESTION 2:

Create a procedure that accepts Customer_ID and displays:

Customer Name
Phone
Email
Account Number
Account Type
Balance

```
396 CREATE PROCEDURE GetCustomerAccountDetails(  
397     IN p_Customer_ID INT  
398 )  
399 BEGIN  
400     SELECT  
401         c.Customer_Name,  
402         c.Phone,  
403         c.Email,  
404         a.Account_No,  
405         a.Account_Type,  
406         a.Balance  
407     FROM Customer c  
408     JOIN Account a  
409     ON c.Customer_ID = a.Customer_ID  
410     WHERE c.Customer_ID = p_Customer_ID;  
411 END //  
412 DELIMITER ;  
413 CALL GetCustomerAccountDetails(101);  
414
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	Customer_Name	Phone	Email	Account_No	Account_Type	Balance
▶	Vaishnavi	9876543210	vaishnavi@gmail.com	10001	Savings	-8000.00

QUESTION 3:

Create a procedure to calculate the total balance of all accounts belonging to a particular customer.

Let's verify

store

down

emas

cted

```
415 DELIMITER //
416
417 CREATE PROCEDURE GetTotalBalance(
418     IN p_Customer_ID INT
419 )
420 BEGIN
421     SELECT
422         p_Customer_ID AS Customer_ID,
423         SUM(Balance) AS Total_Balance
424     FROM Account
425     WHERE Customer_ID = p_Customer_ID;
426 END //
427
428 DELIMITER ;
429 CALL GetTotalBalance(101);
430
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [F1](#)

	Customer_ID	Total_Balance
▶	101	-8000.00